SmartRefactorer: Midterm Report

Brian Guo University of California, Los Angeles CS269-1

Code: github.com/ETwilight/SmartRefactorer

1 Literature review

PSN [1] plans with an executable skill network, repairs failures through trace attribution, and structurally refactors after success. SmartRefactorer (SR) builds on its dependency network and maturity-aware SkillOptimizer. SR targets three concerns: **(1)** reuse instructions do not enforce library-wide architectural conventions, allowing recurring structural problems; **(2)** reusable blocks inside skills lack independent callable interfaces; **(3)** local refactoring opportunities do not by themselves justify immediate migration and validation costs.

SkillZip [2] decomposes skill packages into section graphs, compresses recurring structures, and retrieves dependency-closed context. SR adopts contract-aware component context and dependency analysis. **SkillSmith** [3] reflects on failures and jointly revises skills and tools. SR adopts coordinated edits and reusable failure records. **SpeedRunner** [4] analyzes trajectories to construct and revise executable skills. SR adopts its emphasis on total cost, including skill construction.

Software refactoring [5] changes internal structure while preserving behavior. SR adopts extraction, responsibility moves, explicit dependencies, query/update separation, and inlining, alongside PSN's canonical operations. Further study of *When Should Agents Build Reusable Tools?* [6] will inform maintenance timing.

2 Project idea and approach

Building on PSN's executable network and trace-directed repair, SR coordinates reuse, refactoring, and task development (Figure 1).

When a task arrives, Retrieve exposes related code and records so Plan can determine what to call, extract, or implement. Where extraction is needed, Analyze traces dependencies and callers before Propose coordinates the migration and new implementation under shared interfaces, library conventions, and checks of old behavior.

Develop interleaves migration and new implementation in dependency order: Rewrite exposes old capabilities, CodeGen builds missing behavior around them, and Patch fixes defects. Validation precedes task execution. If either fails, Reflect directs code fixes, proposal revisions, or replanning; affected steps repeat within budget while valid work is retained. After task acceptance and publication checks pass, the candidate is committed. Detect reviews its changes and execution evidence for remaining structural issues. Justified maintenance repeats the same planning, development, validation, and repair process. Otherwise, or after maintenance, the updated library and records pass to the next task.

Long-term token savings. The hypothesis requires recurring, composable capabilities and sufficiently stable interfaces: avoided implementation, debugging, and repeated migration must outweigh planning, extraction, and validation costs over the task stream.

Refactoring cost and quality. Relative to PSN, bounded proposals, retained code and valid checks, and targeted repair should reduce repeated analysis and regeneration. Explicit migration obligations identify missing caller or state updates; diagnosis-guided repair can salvage candidates that would otherwise be rolled back. We will measure tokens per validated refactor, refactor completion, and regressions, alongside cumulative task cost.

Future-demand forecasting. If time permits, forecasts inferred from task history will guide refactoring boundaries, scope, and timing. Their added value will be evaluated against the same workflow without forecasting.

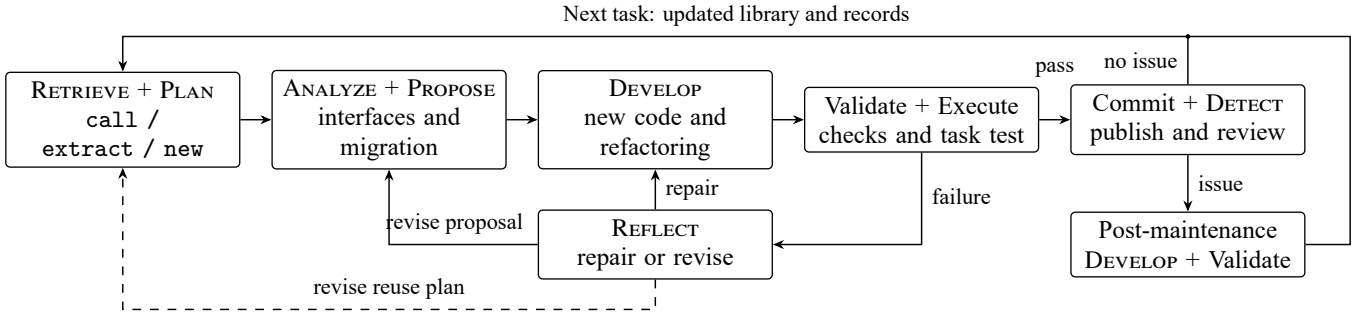


Figure 1: Task development, evidence-directed repair, and incremental post-maintenance.

3 Experimental design

Environments.

1. `techmc_env`: technical Minecraft machines with programmatic acceptance checks. The domain connects to PSN and SpeedRunner while adding engineering demands and reusable collection, transport, timing, and layout structures.
2. `techmc_env_v2`: planned survival-mode production lines and efficiency upgrades, retaining builds and resources, without commands or Litematica.
3. **SlopCodeBench** [7]: agents extend their own code across successive specifications; hidden functional and regression tests assess completion, preservation, and later development cost.

Comparisons and controls. Initial comparisons include baseline, PSN, and SR; later comparisons include SpeedRunner, SkillZip, and SkillSmith. Models, tools, task information, initial libraries, and budgets are matched; every model call is charged, including maintenance and retries. Multiple streams and seeds vary independent task/project order while preserving construction prerequisites and checkpoint order. Future requests remain hidden; acceptance criteria stay fixed.

Evaluation. Metrics include verified task success, input/output tokens per task including failures and maintenance, cumulative cost, capability preservation, and tokens per validated refactor. Refactor-cause analysis covers reuse needs, duplication, interface/state conflicts, repeated architectural violations, and new structural relationships, supported by code changes and execution evidence. Records capture acceptance, rejection, repair cost, and regressions. Comparisons of cause frequencies, recurrence, and pre/post timing across ablations help explain cost and success differences.

Ablations. Each changes one mechanism at fixed model and budget:

1. **Pre-refactoring:** defer extraction until task success; retain reuse of existing callable skills.
2. **Post-maintenance:** disable success-triggered maintenance; retain pre-refactoring and task development.
3. **Repair scope:** retain code repair but disable proposal and reuse-plan revision.
4. **Operation catalog:** restrict refactoring to PSN's five canonical operations; retain SR's controller.

Forecasting, if implemented, will be evaluated separately.

4 Preliminary findings

Pilot experiments have a cap of **30 million tokens per experiment**, counting input and output across the task stream. In current runs, PSN progresses much more slowly than the baseline. The baseline also struggles with complex machines, even the easy-tier iron farm. Naive PSN integration incurs substantial input-token overhead, motivating scoped context, removal of repeated source/history payloads, and on-demand source access. Adapter bugs are also being corrected. Changing implementations and interrupted runs prevent clean attribution to the underlying algorithms; full-scale controlled comparisons remain pending.

SmartRefactorer pilot results: pending.

5 Challenges and open questions

1. Verification and task difficulty. Stateful mechanics complicate regression checks. Hard/Expert machines remain unreached; their mechanics may obscure refactoring gains, while simpler tasks offer little reuse pressure.

2. Fairness and attribution. Baseline may execute ad hoc code while SR builds reusable skills. Tools and acceptance requirements, including code deliverables, must match; SR's maintenance costs count against its budget. Context/adapter optimizations also complicate attribution.

3. Resources. Gemini student verification remains unsuccessful. I am funding API experiments myself and using Codex for development, which limits experimental scale.

References

[1] H. Shi, X. Yuan, and B. Liu. *Evolving Programmatic Skill Networks*. 2026.
[2] X. Tan et al. *SkillZip: Contract-Preserving Graph Compression for Scalable Agent Skill Libraries*. 2026.
[3] Y. Wei et al. *SkillSmith: Co-Evolving Skills and Tools for Self-Improving Agent Systems*. 2026.
[4] Z. Huang et al. *Better, Faster, Stronger: Programmatic Skill Learning Best Reduces Agent Cost*. 2026.
[5] M. Fowler. *Refactoring: Improving the Design of Existing Code*. 2nd ed., Addison-Wesley, 2018.
[6] J. Xu, B. Wilder, and A. Singh. *When Should Agents Build Reusable Tools?* 2026.
[7] G. Orlanski et al. *SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks*. 2026.